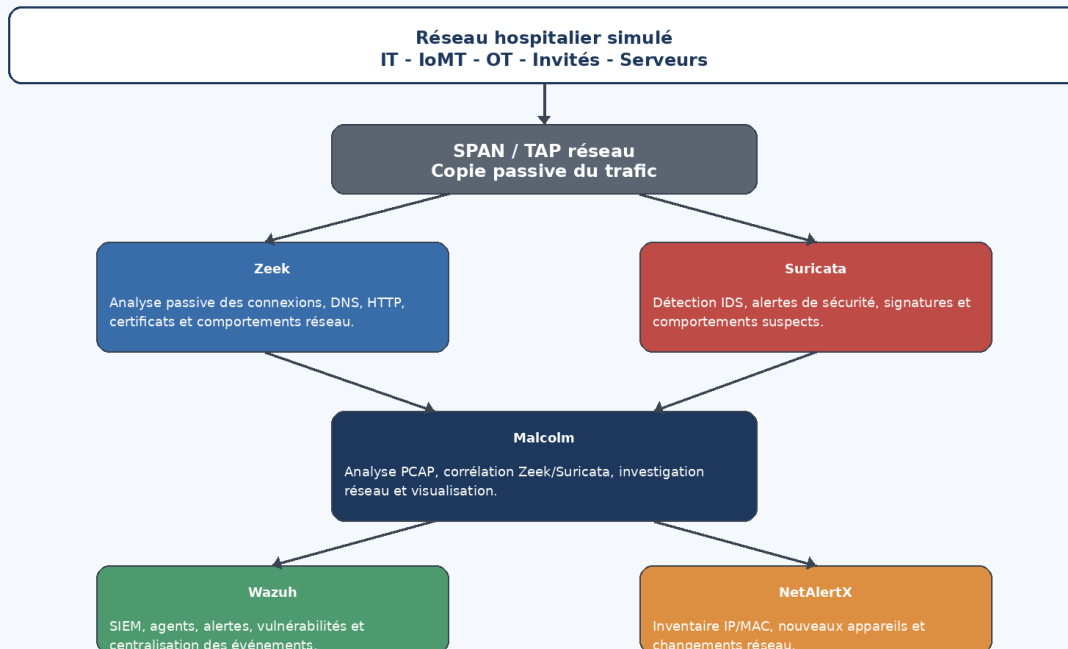


OpenMedShield

Cahier des charges et organisation technique du projet

Architecture logique OpenMedShield

Surveillance passive, visibilité des actifs et centralisation des événements



Stage été 2026 - Département d'informatique de l'Hôpital Montfort

Projet : cybersécurité IoMT, OT et systèmes legacy

Étudiants : Wilsan Waberi et Angoran Laurent Boni

Date : 27/05/2026

Table des matières

1. Résumé exécutif
 2. Contexte et problématique
 3. Objectifs du projet
 4. Cahier des charges
 5. Organisation documentaire du projet
 6. Architecture technique proposée
 7. Outils utilisés et rôle de chaque composant
 8. Ordre d'installation et logique de déploiement
 9. Mise en place de l'environnement
 10. Méthodologie de travail par phase
 11. Inventaire, risques et conformité
 12. Livrables attendus
 13. Conclusion
- Annexe A - Commandes utiles

1. Résumé exécutif

Ce document présente une version structurée et opérationnelle du projet OpenMedShield. Nous y définissons le cahier des charges, l'organisation documentaire, l'architecture technique, les outils open-source retenus, l'ordre d'installation recommandé et les livrables attendus. L'objectif est de transformer les premières recherches en un cadre de travail clair, présentable et exploitable pour la suite du stage.

La démarche retenue repose sur une architecture modulaire permettant d'observer le trafic réseau, de recenser les actifs, de détecter certaines anomalies, de centraliser les événements de sécurité et de produire des recommandations de segmentation. Nous privilégions une surveillance passive afin de limiter les risques de perturbation des équipements biomédicaux et des systèmes legacy.

2. Contexte et problématique

Les environnements hospitaliers modernes regroupent des systèmes informatiques classiques, des technologies opérationnelles, des dispositifs médicaux connectés et des équipements anciens difficiles à mettre à jour. Cette interconnexion améliore les soins et les échanges, mais elle augmente aussi la surface d'attaque. Les systèmes legacy peuvent utiliser des protocoles anciens, des systèmes d'exploitation obsolètes ou des mécanismes d'authentification limités.

Dans ce contexte, le défi consiste à améliorer la visibilité et la surveillance sans intervenir de manière intrusive sur les équipements médicaux. Nous devons donc concevoir une solution de laboratoire qui reproduit les principaux concepts d'une plateforme de cybersécurité IoMT : inventaire, surveillance passive, analyse des communications, détection d'anomalies, centralisation des événements et recommandations de sécurité.

- Manque de visibilité sur les équipements connectés.
- Présence de systèmes biomédicaux legacy difficiles à corriger.
- Risque de mouvements latéraux entre réseaux IT, OT et IoMT.
- Besoin de centraliser les alertes et les journaux de sécurité.
- Nécessité de recommander une segmentation adaptée au contexte hospitalier.

3. Objectifs du projet

L'objectif général est de concevoir un prototype open-source permettant d'améliorer la visibilité, la surveillance et l'analyse des risques dans un environnement hospitalier simulé.

- Recenser les actifs connectés et documenter leurs informations principales.
- Observer les communications réseau de manière passive.
- Identifier les protocoles, flux et comportements inhabituels.
- Mettre en place un SIEM pour centraliser les événements de sécurité.
- Exploiter les journaux Zeek, les alertes Suricata et les analyses Malcolm.
- Structurer les résultats dans des livrables professionnels.
- Produire des recommandations de segmentation, de filtrage et de réduction des risques.

4. Cahier des charges

4.1 Besoin principal

Nous devons mettre en place une architecture open-source permettant de simuler une démarche de cybersécurité appliquée aux réseaux hospitaliers. Cette architecture doit être suffisamment claire pour être présentée aux responsables du stage et suffisamment pratique pour être testée dans un laboratoire.

4.2 Périmètre fonctionnel

- Découverte et inventaire des équipements connectés.
- Analyse passive du trafic réseau.
- Détection d'alertes réseau et d'événements suspects.
- Centralisation des logs et des alertes.
- Analyse de captures PCAP.
- Documentation des risques et recommandations de segmentation.

4.3 Hors périmètre

- Aucune intervention directe sur un équipement médical réel.
- Aucun scan agressif sur un réseau hospitalier de production.
- Aucune modification de configuration clinique ou biomédicale réelle.
- Aucune collecte de données patients réelles.

4.4 Parties prenantes

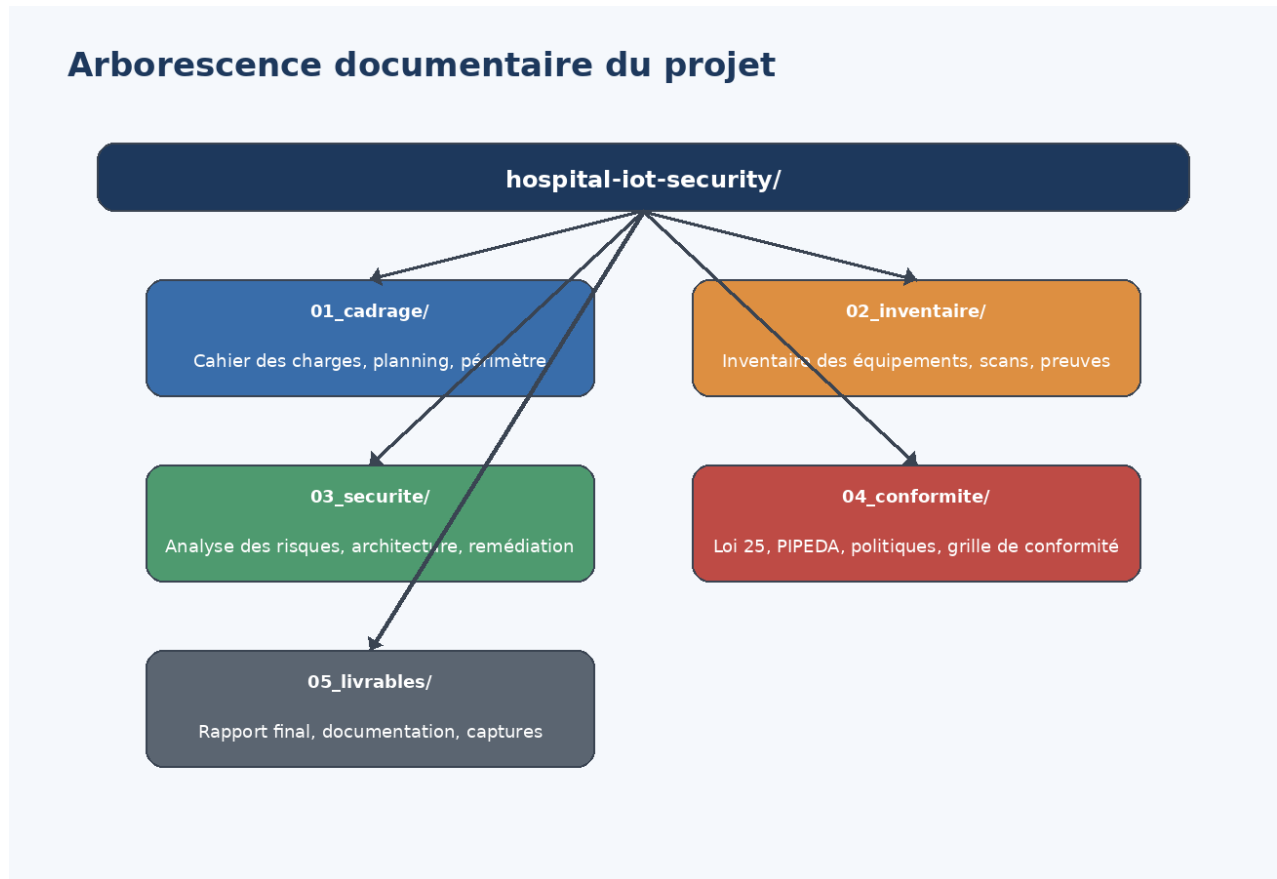
Partie prenante	Rôle attendu
Stagiaires	Recherche, installation, tests, documentation et rapport final.
Responsable de stage	Validation du périmètre, orientation technique et suivi.
Équipe TI / cybersécurité	Contexte opérationnel, contraintes réseau et attentes de sécurité.
Équipe biomédicale	Compréhension des contraintes propres aux dispositifs médicaux.

4.5 Critères de réussite

- Les outils principaux sont installés et accessibles.
- Au moins un agent Wazuh est actif et visible dans le tableau de bord.
- NetAlertX affiche des équipements ou des interfaces réseau détectées.
- Malcolm démarre ses conteneurs et permet l'analyse de captures réseau.
- Le rapport final contient l'architecture, l'inventaire, l'analyse des risques et les recommandations.

- Les limites du prototype sont clairement documentées.

5. Organisation documentaire du projet



L'organisation documentaire permet de séparer le cadrage, l'inventaire, l'analyse de sécurité, la conformité et les livrables. Cette structure facilite la traçabilité du travail et permet de conserver les preuves techniques dans des dossiers dédiés.

Dossier	Contenu	Utilité
01_cadrage	cahier-des-charges.md, planning.xlsx	Définir le périmètre, les objectifs, les acteurs et les échéances.
02_inventaire	inventaire-equipements.xlsx, scans/	Recenser les actifs, conserver les exports et captures d'écran.
03_securite	analyse-risques.md, architecture.drawio	Documenter les risques, l'architecture et les mesures de remédiation.
04_conformite	grille-conformite.xlsx	Associer les risques aux exigences de conformité et aux contrôles proposés.
05_livrables	rapport-final.md	Regrouper les livrables finaux et la documentation présentable.

6. Architecture technique proposée

L'architecture proposée est construite autour d'une logique de surveillance passive. Le trafic réseau est observé à partir d'une copie de flux ou de fichiers PCAP. Les outils spécialisés produisent ensuite des journaux, des alertes et des indicateurs qui peuvent être centralisés dans les tableaux de bord.

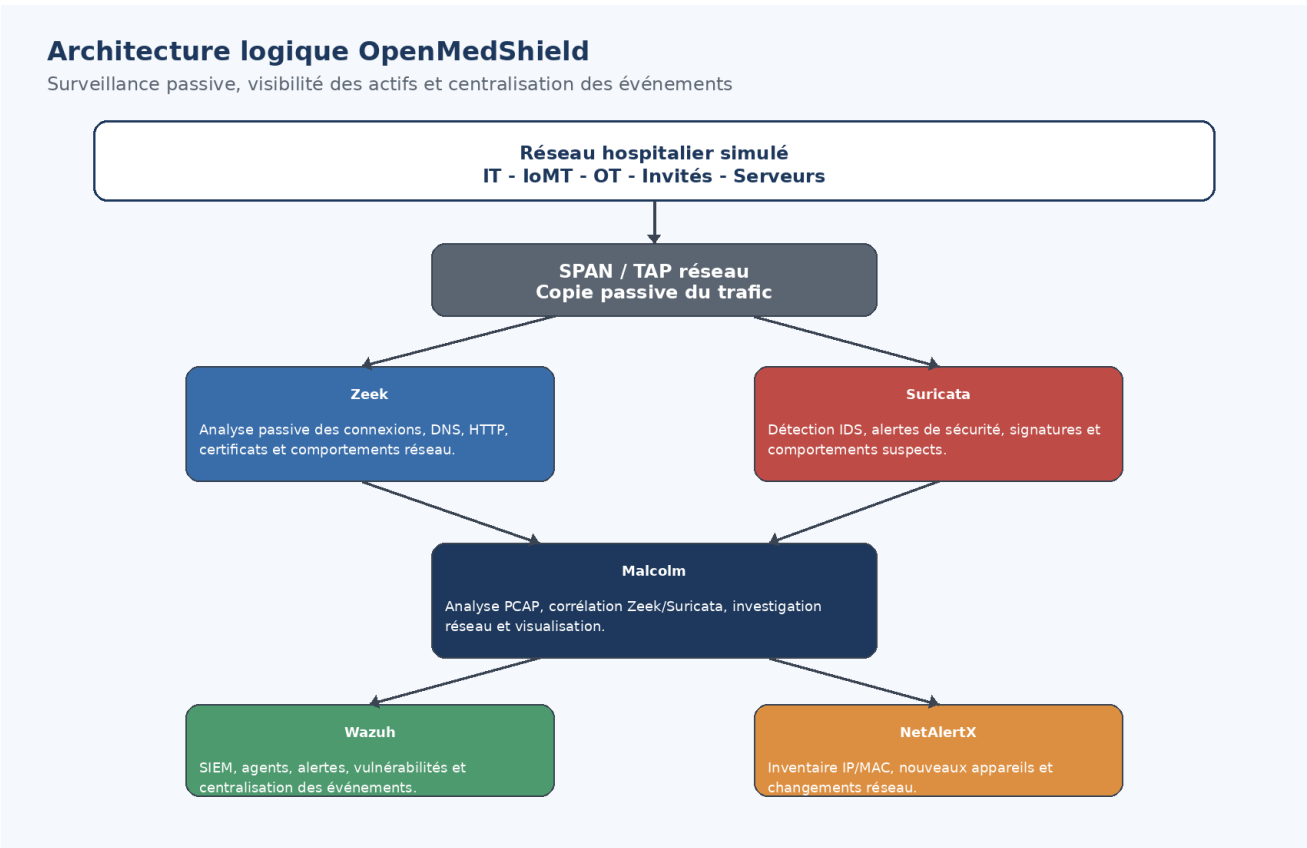


Figure 1 - Architecture logique OpenMedShield.

6.1 Principe général

- NetAlertX fournit une visibilité initiale sur les actifs et changements réseau.
- Zeek analyse passivement les communications et produit des journaux détaillés.
- Suricata détecte des comportements suspects et génère des alertes IDS.
- Malcolm centralise l'analyse réseau, les PCAP, les logs Zeek et les alertes Suricata.
- Wazuh centralise les événements de sécurité, les agents, les vulnérabilités et les alertes.

7. Outils utilisés et rôle de chaque composant

Outil	Rôle	Données produites	Pourquoi nous l'utilisons
Ubuntu VM	Système de laboratoire	Environnement d'exécution	Base Linux stable pour Docker et les outils réseau.
Docker	Moteur de conteneurs	Conteneurs, images, volumes	Simplifie l'installation et l'isolation des

			composants.
NetAlertX	Découverte des actifs	IP, MAC, noms, nouveaux appareils	Permet de constituer l'inventaire réseau.
Wazuh	SIEM et gestion des agents	Alertes, événements, vulnérabilités	Centralise la surveillance et fournit un tableau de bord.
Malcolm	Analyse PCAP et forensic réseau	Sessions, protocoles, métadonnées, visualisations	Facilite l'analyse des captures et des journaux réseau.
Zeek	Analyse passive du trafic	conn.log, dns.log, http.log, ssl.log	Explique les communications réseau en profondeur.
Suricata	IDS réseau	eve.json, fast.log, alertes	Détecte des comportements suspects ou signatures d'attaque.
OpenSearch / Dashboards	Indexation et visualisation	Index, tableaux de bord, recherches	Supporte la recherche et la visualisation dans Wazuh et Malcolm.

8. Ordre d'installation et logique de déploiement



Figure 2 - Ordre logique d'installation des composants.

Nous n'installons pas tous les outils au hasard. L'ordre d'installation doit respecter les dépendances techniques et la logique de validation. Docker doit être installé avant les outils conteneurisés. NetAlertX peut être installé rapidement pour valider la visibilité réseau. Wazuh vient ensuite pour

centraliser les événements. Malcolm est installé sur une VM plus puissante, car il lance plusieurs services lourds.

1. Préparer Ubuntu et vérifier l'espace disque.
2. Installer Docker et Docker Compose.
3. Déployer NetAlertX pour l'inventaire initial.
4. Déployer Wazuh et connecter l'agent Ubuntu.
5. Installer Malcolm sur une VM dédiée ou renforcée.
6. Valider les conteneurs Malcolm : opensearch, dashboards, arkime, zeek, suricata, filebeat.
7. Tester les accès web et les tableaux de bord.
8. Importer ou capturer des PCAP pour valider l'analyse réseau.
9. Documenter les preuves : commandes, captures d'écran, résultats et limites.

9. Mise en place de l'environnement

9.1 Ressources recommandées

Composant	Ressources minimales	Ressources recommandées
VM Wazuh / NetAlertX	6 à 8 Go RAM, 2 CPU, 80 Go disque	10 à 12 Go RAM, 4 CPU, 100 Go disque
VM Malcolm	8 Go RAM, 4 CPU, 80 Go disque	12 à 16 Go RAM, 4 à 6 CPU, 120 Go disque
Stockage PCAP	20 Go libres	50 Go ou plus selon les captures

9.2 Validation des composants installés

Composant	Commande ou preuve	Résultat attendu
Docker	<code>docker --version / docker ps</code>	Docker répond et liste les conteneurs.
NetAlertX	<code>http://localhost:20211</code>	Tableau de bord accessible.
Wazuh	<code>https://localhost</code>	Dashboard accessible et agent actif.
Agent Wazuh	<code>sudo systemctl status wazuh-agent</code>	active (running).
Malcolm	<code>docker ps</code>	Conteneurs Malcolm démarrés.
Malcolm URLs	<code>./scripts/print-urls</code>	Liens web des services Malcolm affichés.

10. Méthodologie de travail par phase

Phase	Travail à réaliser	Livrables
Phase 1 - Analyse	Comprendre l'environnement hospitalier générique, les zones	Synthèse d'architecture et

	réseau et les systèmes legacy.	contexte.
Phase 2 - Risques	Identifier les vulnérabilités, risques, contraintes et scénarios de menace.	Analyse des risques.
Phase 3 - Conception	Définir l'architecture open-source, les outils et l'ordre d'installation.	Cahier des charges, architecture, plan de déploiement.
Phase 4 - Prototype	Installer les composants, valider les tableaux de bord et tester les données.	Captures d'écran, commandes, résultats.
Phase 5 - Recommandations	Interpréter les résultats et proposer des mesures de sécurisation.	Rapport final et recommandations.

11. Inventaire, risques et conformité

11.1 Inventaire des équipements

Le fichier d'inventaire doit devenir la référence centrale des actifs observés dans le laboratoire. Chaque équipement détecté doit être associé à une zone réseau, une criticité et des protocoles observés.

Champ	Description
Nom de l'équipement	Nom court ou nom détecté par l'outil.
Adresse IP	Adresse IPv4 ou IPv6 observée.
Adresse MAC	Identifiant réseau de l'équipement.
Type	Poste, serveur, IoMT simulé, routeur, conteneur, autre.
Zone réseau	IT, IoMT, OT, invité, serveur, laboratoire.
Criticité	Faible, moyenne, élevée, critique.
Protocoles	DNS, HTTP, HTTPS, SMB, DICOM, HL7 simulé, autre.
Commentaires	Informations utiles pour l'analyse et la remédiation.

11.2 Analyse des risques

- Risque de propagation latérale entre segments mal isolés.
- Risque d'exposition de systèmes legacy non corrigés.
- Risque de communications inutiles entre équipements.
- Risque d'alertes non traitées faute de centralisation.
- Risque de mauvaise visibilité sur les actifs inconnus.

11.3 Conformité

La conformité n'est pas traitée comme une simple formalité. Elle sert à relier les mesures techniques aux exigences de protection des données, de traçabilité, d'accès contrôlé et de réduction des risques. La grille de conformité pourra couvrir la Loi 25, PIPEDA, HIPAA comme référence comparative, NIST 800-53 et les principes Zero Trust.

12. Livrables attendus

Livrable	Format	Contenu attendu
Cahier des charges	Markdown / PDF	Objectifs, périmètre, contraintes, outils, critères de réussite.
Inventaire des équipements	Excel	IP, MAC, type, zone, criticité, protocoles observés.
Analyse des risques	Markdown / PDF	Vulnérabilités, impacts, mesures de mitigation.
Architecture	Draw.io / image	Schéma logique de l'architecture OpenMedShield.
Grille de conformité	Excel	Exigences, contrôles, preuves et commentaires.
Rapport final	PDF	Synthèse complète, résultats, limites et recommandations.

13. Conclusion

Cette version structurée transforme les premières recherches en un cadre professionnel et exploitable. Elle définit clairement le cahier des charges, l'organisation documentaire, l'architecture technique, l'ordre d'installation des outils et les livrables.

La démarche proposée montre que le projet ne consiste pas seulement à installer des logiciels. Il s'agit d'une approche complète : cadrer le besoin, observer l'environnement, construire une architecture, tester les composants, analyser les résultats et proposer des recommandations adaptées aux environnements hospitaliers.

Annexe A - Commandes utiles

A.1 Vérifier Docker

```
docker --version
docker compose version
docker ps
docker ps -a
```

A.2 NetAlertX

```
cd ~/OpenMedShield/netaalertx
docker compose up -d
docker logs netaalertx
```

A.3 Wazuh Agent

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent
```

A.4 Malcolm

```
cd ~/OpenMedShield/Malcolm
./scripts/configure
./scripts/start
docker compose --profile malcolm up -d
docker ps
./scripts/print-urls
```